

Multi – level Linked List

–Print List(Main List and Child)

–Time Complexity Calculation using Recursive Tree Method

Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}

...

case 9:
    cout << "Current List Structure: " << endl;
    printList(head);
    break;
```

We have seen earlier :

$$2T\left(\frac{N-1}{2}\right) + c \cdot \log_2 n, \text{ Base Case: } T(0) = 1$$

[Wrong]

Also, some may consider ,

$$2T\left(\frac{N-1}{2}\right) + c \cdot n, \text{ Base Case: } T(0) = 1$$

Note: here $c \cdot n$, and n is the problem size. Very important, must recognize.

Recursion Step	No. of problems	Problem Size	Node's Level (k)	Work/Cost Per Problem (c · n)
0	1	n	0	c × N
1	2	$\frac{n-1}{2}$	1	$c \times \left(\frac{N-1}{2}\right)$
2	4	$\frac{n-3}{4}$	2	$c \times \left(\frac{n-3}{4}\right)$
3	8	$\frac{n-7}{8}$	3	$c \times \left(\frac{n-7}{8}\right)$
· ·	· ·	· ·	· ·	· ·
k	2^k	$\frac{N - (2^k - 1)}{2^k}$	k	$c \times \left(\frac{N - (2^k - 1)}{2^k}\right)$
log₂(N + 1) (Base Case)	N + 1	$\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}}$ = 0 (Base Case)	N/A	T(0) = 1

$$\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}} \text{ (Base Case)}$$

$$= \frac{N - (N + 1 - 1)}{N + 1}$$

$$= \frac{N - N - 1 + 1}{N + 1}$$

$$= \frac{0}{N+1} = 0$$

Also ,no. of problems : 2^k , when $k = \log_2(N+1) \Rightarrow 2^{\log_2(N+1)} \Rightarrow N+1$

<i>Work done = Work per Problem × No. of Problems</i>
$1 \times cN = cN$
$2 \left(c \times \left(\frac{N-1}{2} \right) \right) = c(N-1)$
$4 \left(c \times \left(\frac{n-3}{4} \right) \right) = c(n-3)$
$8 \left(c \times \left(\frac{n-7}{8} \right) \right) = c(n-7)$
.
$2^k \left(c \times \left(\frac{N - (2^k - 1)}{2^k} \right) \right) = c(N - (2^k - 1))$
$(N+1) \times T(0)$

$$T(N) = cN + c(N - 1) + c(n - 3) + \dots + c(N - (2^k - 1)) \\ + (N + 1) \times T(0)$$

$$T(N) = c(N - (2^0 - 1)) + c(N - (2^1 - 1)) + c(n - (2^2 - 1)) \\ + \dots + c(N - (2^k - 1)) + (N + 1) \times T(0)$$

$$T(N) = c \times \sum_{i=0}^k (N - (2^i - 1)) + (N + 1) \times T(0)$$

Lets take 1st term:

$$c \times \sum_{i=0}^k (N - (2^i - 1)) = c \times \left(\sum_{i=0}^k N - \sum_{i=0}^k (2^i - 1) \right)$$

Here:

$$\sum_{i=0}^k N = \underbrace{N + N + N + \dots + N}_{\downarrow}$$

$k + 1 \text{ times.}$

$$\therefore \sum_{i=0}^k N = (k + 1)N.$$

$$\therefore c \times \left(\sum_{i=0}^k N - \sum_{i=0}^k (2^i - 1) \right) \Rightarrow c \times \left((k+1)N - \sum_{i=0}^k (2^i - 1) \right)$$

$$\Rightarrow c \times \left((k+1)N - \left(\sum_{i=0}^k 2^i - \sum_{i=0}^k 1 \right) \right)$$

$$\sum_{i=0}^k 1 = \underbrace{1 + 1 + 1 + \dots + 1}_{\substack{\downarrow \\ \boxed{k+1 \text{ times.}}}}$$

$$\Rightarrow c \times \left((k+1)N - \left(\sum_{i=0}^k 2^i - (k+1) \times 1 \right) \right)$$

$$\Rightarrow c \times \left((k+1)N - \left(\sum_{i=0}^k 2^i - (k+1) \right) \right)$$

Now,

$$\sum_{i=0}^k 2^i = 1 + 2 + 4 + \dots + 2^k$$

Let,

$$S = 1 + 2 + 4 + \dots + 2^k$$

And , multiplying both sides by 2:

$$\begin{aligned} 2S &= 2 + 4 + 8 + \dots + (2^{k-1} \times 2) + 2^k \times 2 \\ &= 2 + 4 + 8 + \dots + (2^{k-1+1}) + 2^{k+1} \\ &= 2 + 4 + 8 + \dots + (2^k) + 2^{k+1} \end{aligned}$$

Now,

$$\begin{aligned} 2S - S &= (0 - 1) + (2 - 2) + (4 - 4) + (8 - 8) + (2^k - 2^k) \\ &\quad + (2^{k+1} - 0) \end{aligned}$$

$$\therefore S = -1 + 2^{k+1}$$

$$\text{or, } S = 2^{k+1} - 1$$

$$\therefore \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

$$\Rightarrow c \times \left((k+1)N - (2^{k+1} - 1 - (k+1)) \right)$$

$$\Rightarrow c \times \left(N(k+1) - (2^{k+1} - 1 - k - 1) \right)$$

$$\Rightarrow c \times \left(N(k+1) - (2^{k+1} - k - 2) \right)$$

$$\Rightarrow c \times (N(k+1) - 2^{k+1} + k + 2)$$

we know, $T(0)$ is our base case .

The tree stops growing (it hits the base case) when the problem size reaches 0.

So, to find the final level k , we set our pattern equal to 0 and solve for k :

$$\frac{N - (2^k - 1)}{2^k} = 0$$

Multiplying 2^k on both the sides:

$$\Rightarrow N - (2^k - 1) = 0$$

Moving the $2^k - 1$ to the other side, we get:

$$\Rightarrow N = 2^k - 1$$

Add 1 to both sides to isolate the exponential term:

$$\Rightarrow N + 1 = 2^k$$

Take the base - 2 logarithm of both sides.

Because the base of our exponent is 2 (from 2^k), we use the base - 2 logarithm ($\log_2$) to target it.

$$\begin{aligned}\Rightarrow \log_2(N+1) &= \log_2 2^k \\ \Rightarrow \log_2(N+1) &= k \times \log_2 2 \\ \Rightarrow \log_2(N+1) &= k [\log_a a = 1]\end{aligned}$$

$$\begin{aligned}\therefore k &= \log_2(N+1) \\ \Rightarrow c \times (N(\log_2(N+1) + 1) - 2^{\log_2(N+1)+1} + \log_2(N+1) + 2) \\ \Rightarrow c \times (N \log_2(N+1) + N - 2^{\log_2(N+1)} \times 2 + \log_2(N+1) + 2) \\ \Rightarrow c \times (N \log_2(N+1) + N - (N+1) \times 2 + \log_2(N+1) + 2) \\ \Rightarrow c \times (N \log_2(N+1) + N - (2N+2) + \log_2(N+1) + 2) \\ \Rightarrow c \times (N \log_2(N+1) + N - 2N - 2 + \log_2(N+1) + 2) \\ \Rightarrow c \times (N \log_2(N+1) + \log_2(N+1) - N) \\ \Rightarrow c \times ((N+1) \log_2(N+1) - N)\end{aligned}$$

$$T(N) = c \times \sum_{i=0}^k (N - (2^i - 1)) + (N+1) \times T(0)$$

$$T(N) = c \times ((N+1) \log_2(N+1) - N) + (N+1) \times T(0)$$

As $T(0)$ is 1, then :

$$T(N) = c \times ((N+1) \log_2(N+1) - N) + (N+1) \times 1$$

$$T(N) = c \times ((N + 1) \log_2(N + 1) - N) + (N + 1)$$

$$\Rightarrow T(N) = \Theta(N \log_2 N)$$

Its looks like ,

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$

$$\Theta(N \log_2 N)$$

=

$$2T\left(\frac{N-1}{2}\right) + c \cdot n, \text{Base Case: } T(0) = 1$$

$$\Theta(N \log_2 N)$$

But its not the case .

Remember I have said :

Note: here $c.n$, and n is the problem size. Very important, must recognize.

Similarly ,

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$

Here, $c_1 \times k + c_2$, where $k = \log_2(n + 1)$ is the Problem size.

Therefore,

(N)	$(N + 1)$	$k = \log_2(n + 1)$
10	11	3.4594
11	12	3.5850
12	13	3.7004
13	14	3.8074
14	15	3.9069
15	16	4.0000

Therefore ,no way:

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$
$$\approx 2T\left(\frac{N-1}{2}\right) + c \cdot n, \text{Base Case: } T(0) = 1$$

Hence,

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$

$$\Theta(N \log_2 N)$$

$\neq$

$$2T\left(\frac{N-1}{2}\right) + c \cdot n, \text{Base Case: } T(0) = 1$$

$$\Theta(N \log_2 N)$$

Hence ,it is strictly :

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{Base Case: } T(0) = 1.$$

No generalization can take place.
